

“Karte Dashboard” back-end project documentation

Prepared by: Tuvshin - Erdene.E

Backend System Architecture	3
Overview	3
The Three-Layer Pattern (MCR)	3
1. Routes (/routes) — The Gatekeepers	3
2. Controllers (/controllers) — The Orchestrators	3
3. Models (/models) — The Data Engine	3
Core System Features	4
Real-Time Communication (Socket.io)	4
The Patient State Machine	4
Autonomous Bottleneck Detection	4
Tech Stack Summary	4
Typical Data Flow	5
Hospital Monitoring System: API Documentation	6
Overview	6
Authentication	6
Real-Time WebSockets (Socket.io)	6
1. Connection	6
2. Joining Rooms (Crucial)	6
3. Events Emitted by Server	7
Hospital Endpoints (/hospital)	7
Event Payload Example (POST /events):	8
Support & Monitoring Endpoints	8
Notifications (/support/notifications)	8
Support Requests (/support/requests)	8
Business Logic Note: The 3-Timestamp Flow	8
Troubleshooting	9

Backend System Architecture

Overview

The backend is built with **Node.js** and **Express**, using **PostgreSQL** as the data engine. Its primary purpose is to act as a "Real-Time Traffic Controller" for the hospital, managing the transition of patients between various medical states while monitoring for delays.

The Three-Layer Pattern (MCR)

1. Routes (/routes) — The Gatekeepers

Routes define the entry points (URLs) of the application. They are responsible for:

- **Mapping:** Linking a specific URL (e.g., POST /api/hospital/events) to a function in a Controller.
- **Security:** Applying **Middleware** like `authMiddleware` to check JWT tokens before allowing access.
- **Special Logic:** Handling exceptions, such as allowing the Python Simulator to post events without a standard user login.

2. Controllers (/controllers) — The Orchestrators

Controllers act as the "middlemen." When a request hits a route, the controller:

- **Extracts Data:** Pulls information from the request body (`req.body`) or URL parameters (`req.params`).
- **Calls the Model:** Tells the Model what data to fetch or update in the database.
- **Broadcasts Updates:** This is where the **Socket.io** integration lives. After a successful database change, the controller emits events (`req.io.emit`) to update the frontend dashboard instantly.
- **Sends Responses:** Sends the final JSON response and HTTP status codes (201 for success, 202 for queueing, etc.) back to the client.

3. Models (/models) — The Data Engine

The Models contain the **raw intelligence** and SQL queries. They are the only part of the code that talks directly to PostgreSQL.

- **Complex Logic:** Our models handle advanced SQL, such as checking if a doctor is "busy" by looking at other active events.
 - **Transactions:** For critical actions like processEvent, the Model uses BEGIN and COMMIT commands. This ensures that if one part of the process fails (like creating a visit), the whole action is rolled back to prevent "broken" data.
 - **Data Transformation:** They format raw database rows into clean objects for the rest of the system to use.
-

Core System Features

Real-Time Communication (Socket.io)

Instead of the dashboard "asking" the server for updates every few seconds, the server "pushes" updates to the dashboard.

- When a patient starts a step, the **Controller** emits a HOSPITAL_UPDATE.
- The dashboard receives this and updates its counters immediately.

The Patient State Machine

The backend manages a three-stage lifecycle for every medical step:

1. **Waiting:** Patient is logged in the system but no doctor is free.
2. **In Progress:** A doctor has been assigned and is currently with the patient.
3. **Completed:** The treatment is done and the resource (doctor) is released.

Autonomous Bottleneck Detection

The backend automatically monitors the time elapsed between "Entering" and "Exiting" a room.

- If **Time > Mid-Threshold**, it flags a **Warning**.
 - If **Time > High-Threshold**, it flags a **Critical** alert.
 - These are saved in the notifications table and broadcasted to the Admin room via Sockets.
-

Tech Stack Summary

- **Framework:** Express.js (Node.js)
- **Database:** PostgreSQL
- **Real-time:** Socket.io

- **Auth:** JWT (JSON Web Tokens)
 - **Migration:** db-migrate (for version-controlled schema changes)
-

Typical Data Flow

1. **Request:** Simulator sends POST /api/hospital/events (START).
2. **Route:** Route receives it and passes it to HospitalController.handleEvent.
3. **Model:** Model runs SQL to find a free doctor. It updates the DB status to in_progress.
4. **Socket:** Controller sees the update and calls io.emit('HOSPITAL_UPDATE').
5. **Dashboard:** The Frontend React app hears the event and increments the "Active Patients" count on the screen.
6. **Response:** Controller sends 201 Created back to the Simulator.

Hospital Monitoring System: API Documentation

Overview

This API manages patient flow, staff assignments, and real-time bottleneck detection. It supports standard REST calls and real-time updates via Socket.io.

Base URL: `http://localhost:8080/api`

Authentication

Most endpoints are protected by JWT middleware.

1. **Login:** Send credentials to `/auth/login`.
 2. **Usage:** Include the returned token in the header: `Authorization: Bearer <token>`.
 3. **Simulator Bypass:** The `/hospital/events` endpoint is specifically designed to allow traffic from our Python Simulator.
-

Real-Time WebSockets (Socket.io)

We use WebSockets to provide instant dashboard updates without page refreshes.

1. Connection

Connect to the root server:

```
code JavaScript
import { io } from "socket.io-client";
const socket = io("http://localhost:8080");
```

2. Joining Rooms (Crucial)

To receive data, a client **must** join a room.

- **Hospital Room:** `socket.emit('join_hospital', hospitalId)`
 - *Use this for the main dashboard to only get updates for your specific facility.*
-
- **Admin Room:** `socket.emit('join_admin')`

- Use this for the global operator view to see all notifications.

3. Events Emitted by Server

The frontend should listen for these:

| Event Name | Data Payload | Description |

| :--- | :--- | :--- |

| HOSPITAL_UPDATE | { action, patient_identifier, hospital_step_id, status } | Triggered every time a patient moves (START/END). |

| BOTTLENECK_ALERT | { type, message, hospital_step_id } | Triggered when a patient exceeds a time threshold. |

Hospital Endpoints (/hospital)

Method	Endpoint	Description
GET	/	List all hospitals.
GET	/:id/snapshot	Current counts of waiting vs in_progress patients per step.
GET	/:id/service	List all services (Cardiology, ER, etc.) for a hospital.
GET	/:id/service/:serviceId/steps	Get the ordered sequence of steps for a service.
GET	/:id/staff	List all doctors working at this hospital.
GET	/steps/:stepId/staff	List doctors authorized to perform a specific step.
POST	/events	Main Traffic Port. Used to move patients between steps.

Event Payload Example (POST /events):

code JSON

```
{
  "patient_identifier": "AUTO-1234",
  "hospital_step_id": "uuid-step-id",
  "action": "START", // or "END"
  "staff_id": "uuid-doctor-id", // Optional: system picks random if null
  "custom_timestamp": "2026-06-26T10:00:00Z" // Optional: for backdating
}
```

Support & Monitoring Endpoints

Notifications (/support/notifications)

- **GET** / : Fetch all bottleneck alerts.
- **PATCH** /:id/read : Mark an alert as resolved.

Support Requests (/support/requests)

- **POST** / : Create a new help ticket (e.g., doctor requesting backup).
 - **PATCH** /:id : Update status (open, in_progress, resolved).
-

Business Logic Note: The 3-Timestamp Flow

For the frontend team, please note that every visit_event follows this cycle:

1. **Arrived:** Created when START is called but all doctors are busy. (status: waiting).
2. **Entered:** Updated when a doctor becomes free. (status: in_progress).
3. **Exited:** Updated when END is called. (status: completed).

Wait Time = Entered - Arrived.

Process Time = Exited - Entered.

Troubleshooting

- **Database:** If queries are slow, check that `hospital_step_id` and `status` are indexed.
- **Sockets:** If you aren't receiving updates, ensure you called `join_hospital` with a valid UUID string immediately after connecting.
- **Simulator:** If the simulator is blocked, ensure `DISABLE_AUTH=true` is set in the `.env` file of the backend.